บทที่ 5: การทำสำเนาข้อมูล (Replication)

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 18–20 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่ห้า ว่าด้วยการทำสำเนาข้อมูล หรือ Replication

บทนี้เป็นบทแรกของภาคที่สอง ซึ่งพูดถึงข้อมูลแบบกระจาย หมายถึงข้อมูลที่เก็บไว้บนหลายเครื่อง ที่เชื่อมต่อกันผ่านเครือข่าย

การทำสำเนาข้อมูล คือการเก็บข้อมูลชุดเดียวกันไว้หลายเครื่อง แต่ละเครื่องที่เก็บสำเนาเรียกว่า replica เหตุผลที่ต้องทำมีสามข้อ

ข้อแรก เพื่อให้ข้อมูลอยู่ใกล้ผู้ใช้ เช่น มีสำเนาไว้ทั้งในเอเชียและยุโรป ผู้ใช้แต่ละที่จะได้คำตอบเร็วขึ้น

ข้อสอง เพื่อให้ระบบยังทำงานต่อได้ แม้บางเครื่องจะเสีย

ข้อสาม เพื่อเพิ่มจำนวนเครื่องที่ช่วยกันตอบคำขออ่านข้อมูล ระบบจะรับผู้ใช้ได้มากขึ้น

ในบทนี้ เราสมมติว่าข้อมูลทั้งหมดใส่ในเครื่องเดียวได้ ส่วนกรณีที่ข้อมูลใหญ่เกินเครื่องเดียว ต้องแบ่งข้อมูลเป็นส่วน ๆ ซึ่งเป็นเรื่องของบทถัดไป

ถ้าข้อมูลไม่เคยเปลี่ยน การทำสำเนาก็ง่ายมาก แค่คัดลอกครั้งเดียวจบ ความยากทั้งหมดอยู่ที่การจัดการเมื่อข้อมูลเปลี่ยน วิธีหลักมีสามแบบ คือ แบบมีผู้นำคนเดียว แบบมีผู้นำหลายคน และแบบไม่มีผู้นำ เราจะไปดูทีละแบบครับ

ตอนที่ 2: แบบมีผู้นำคนเดียว

แบบนี้ใช้กันมากที่สุด หลักการเป็นแบบนี้

- สำเนาหนึ่งได้รับเลือกให้เป็นผู้นำ หรือ leader การเขียนข้อมูลทั้งหมดต้องส่งมาที่ผู้นำ
- สำเนาอื่นเป็นผู้ตาม หรือ follower เมื่อผู้นำเขียนข้อมูลใหม่ ก็ส่งรายการเปลี่ยนแปลงไปให้ผู้ตามทุกตัว ผู้ตามจะทำตามลำดับเดียวกัน
- การอ่านข้อมูลอ่านได้จากทั้งผู้นำและผู้ตาม แต่การเขียนทำได้ที่ผู้นำเท่านั้น

ฐานข้อมูลที่ใช้วิธีนี้ เช่น PostgreSQL, MySQL, SQL Server และ MongoDB ระบบส่งข้อความอย่าง Kafka และ RabbitMQ ก็ใช้วิธีนี้ด้วย

แบบรอ กับ แบบไม่รอ

เมื่อผู้นำส่งข้อมูลให้ผู้ตาม มีสองทางเลือก

ทางแรกคือแบบรอ หรือ synchronous ผู้นำจะรอจนผู้ตามยืนยันว่าบันทึกแล้ว ถึงจะบอกผู้ใช้ว่าเขียนสำเร็จ ข้อดีคือผู้ตามมีข้อมูลล่าสุดแน่นอน ข้อเสียคือ ถ้าผู้ตามช้าหรือล่ม ผู้นำก็เขียนอะไรต่อไม่ได้เลย

ทางที่สองคือแบบไม่รอ หรือ asynchronous ผู้นำบอกผู้ใช้ว่าสำเร็จทันที แล้วค่อยส่งข้อมูลให้ผู้ตามทีหลัง ข้อดีคือเร็วและไม่ติดขัด ข้อเสียคือ ถ้าผู้นำเสียก่อนส่งข้อมูลออกไป ข้อมูลที่บอกผู้ใช้ว่าบันทึกแล้วอาจหายไป

ในทางปฏิบัติ การให้ผู้ตามทุกตัวเป็นแบบรอนั้นทำไม่ได้ เพราะแค่เครื่องเดียวล่มก็ทำให้ระบบทั้งระบบหยุด หลายระบบจึงให้ผู้ตามแบบรอแค่หนึ่งตัว ที่เหลือเป็นแบบไม่รอ วิธีนี้เรียกว่า semi-synchronous ส่วนระบบอื่นอีกมากก็ใช้แบบไม่รอทั้งหมด แม้จะเสี่ยงข้อมูลหายก็ตาม

การเพิ่มผู้ตามตัวใหม่

จะคัดลอกไฟล์ข้อมูลตรง ๆ ไม่ได้ เพราะข้อมูลเปลี่ยนอยู่ตลอด ขั้นตอนที่ใช้จริงคือ

- ถ่ายภาพข้อมูลของผู้นำ ณ เวลาหนึ่ง เรียกว่า snapshot
- คัดลอกภาพนั้นไปที่ผู้ตามตัวใหม่
- ผู้ตามขอรายการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นหลังจากถ่ายภาพ
- เมื่อทำตามรายการจนครบ ผู้ตามก็ตามทันผู้นำ แล้วรับการเปลี่ยนแปลงต่อไปตามปกติ

ตอนที่ 3: เมื่อเครื่องเสีย

ถ้าผู้ตามเสีย เรื่องนี้ไม่ยาก ผู้ตามจำได้ว่าทำรายการเปลี่ยนแปลงไปถึงไหนแล้ว พอกลับมาก็ขอรายการที่ขาดไปจากผู้นำแล้วทำต่อจนตามทัน

ถ้าผู้นำเสีย เรื่องนี้ยากกว่ามาก ต้องเลือกผู้ตามตัวหนึ่งขึ้นมาเป็นผู้นำแทน เรียกว่า failover ขั้นตอนคือ

- ตรวจให้แน่ใจว่าผู้นำเสียจริง ปกติใช้วิธีจับเวลา ถ้าผู้นำไม่ตอบนานเกินที่กำหนด ก็ถือว่าเสีย
- เลือกผู้นำคนใหม่ ซึ่งควรเป็นผู้ตามที่มีข้อมูลล่าสุดที่สุด
- ตั้งค่าระบบให้ส่งการเขียนไปหาผู้นำคนใหม่ และถ้าผู้นำเก่ากลับมา ต้องยอมเป็นผู้ตาม

สิ่งที่อาจผิดพลาดตอนเปลี่ยนผู้นำ

ปัญหาแรก ถ้าใช้แบบไม่รอ ผู้นำคนใหม่อาจยังไม่ได้รับข้อมูลชุดสุดท้ายจากผู้นำเก่า ข้อมูลชุดนั้นจึงมักถูกทิ้งไป

หนังสือยกเหตุการณ์จริงที่ GitHub ผู้ตามที่ข้อมูลล้าหลังได้เป็นผู้นำ แล้วนำเลขรหัสที่ผู้นำเก่าเคยใช้ไปแล้วมาใช้ซ้ำ ซึ่งเลขพวกนี้ถูกใช้ในอีกระบบหนึ่งด้วย ผลคือข้อมูลส่วนตัวของผู้ใช้บางคนไปปรากฏให้ผู้ใช้คนอื่นเห็น

ปัญหาที่สอง คือมีผู้นำสองคนพร้อมกัน เพราะต่างฝ่ายต่างคิดว่าตัวเองเป็นผู้นำ ทั้งคู่รับการเขียนข้อมูล ข้อมูลจึงขัดแย้งกันและอาจเสียหาย สถานการณ์นี้เรียกว่า split brain หรือสมองแยก บางระบบแก้ด้วยการตัดไฟเครื่องหนึ่งทิ้งไปเลย

ปัญหาที่สาม คือตั้งเวลาเท่าไรดี ถ้าสั้นเกินไป แค่เครือข่ายช้าชั่วคราวก็เปลี่ยนผู้นำโดยไม่จำเป็น ซึ่งยิ่งทำให้ระบบที่กำลังหนักอยู่แล้วแย่ลงไปอีก ถ้านานเกินไป ระบบก็หยุดให้บริการนาน

เพราะปัญหาเหล่านี้ ทีมดูแลระบบหลายแห่งจึงเลือกเปลี่ยนผู้นำด้วยมือ แม้ระบบจะทำอัตโนมัติได้ก็ตาม

ตอนที่ 4: ผู้นำส่งรายการเปลี่ยนแปลงอย่างไร

มีวิธีหลักสี่แบบ

แบบแรก ส่งคำสั่งไปตรง ๆ ผู้นำส่งคำสั่ง SQL ที่ได้รับไปให้ผู้ตามรันซ้ำ ฟังดูง่าย แต่มีปัญหา เช่น คำสั่งที่ขอเวลาปัจจุบัน หรือขอเลขสุ่ม จะได้ผลไม่เหมือนกันในแต่ละเครื่อง

แบบที่สอง ส่งไฟล์บันทึกที่เขียนลงดิสก์ คือส่งไฟล์ write-ahead log ที่เราพูดถึงในบทที่สามไปให้ผู้ตาม PostgreSQL ใช้วิธีนี้ ข้อเสียคือไฟล์นี้บันทึกในระดับไบต์บนดิสก์ จึงผูกติดกับเวอร์ชันของฐานข้อมูล ถ้าผู้นำกับผู้ตามใช้คนละเวอร์ชัน อาจใช้กันไม่ได้ การอัปเกรดจึงมักต้องหยุดระบบ

แบบที่สาม ส่งรายการเปลี่ยนแปลงระดับแถว หรือ logical log บอกว่าแถวไหนถูกเพิ่ม ลบ หรือแก้เป็นค่าอะไร MySQL ใช้วิธีนี้ รูปแบบนี้ไม่ขึ้นกับเวอร์ชันของฐานข้อมูล และโปรแกรมภายนอกอ่านได้ง่าย เช่น ส่งการเปลี่ยนแปลงต่อไปให้คลังข้อมูล ซึ่งเรียกว่า change data capture

แบบที่สี่ ให้โค้ดของแอปพลิเคชันจัดการเอง โดยใช้สิ่งที่เรียกว่า trigger ให้ฐานข้อมูลรันโค้ดอัตโนมัติทุกครั้งที่ข้อมูลเปลี่ยน วิธีนี้ยืดหยุ่นที่สุด แต่ช้ากว่าและมีโอกาสเกิดบั๊กมากกว่า

ตอนที่ 5: ปัญหาเมื่อสำเนาตามไม่ทัน

ถ้าใช้แบบไม่รอ ผู้ตามอาจล้าหลังผู้นำไปบ้าง ปกติแค่เสี้ยววินาที แต่ถ้าระบบหนักหรือเครือข่ายมีปัญหา อาจล้าหลังเป็นนาที

ถ้าหยุดเขียนข้อมูลสักพัก ผู้ตามก็จะตามทันในที่สุด เรียกว่า eventual consistency หรือความสอดคล้องในท้ายที่สุด แต่คำว่าท้ายที่สุดนี้ไม่ได้บอกว่านานแค่ไหน และระหว่างนั้นผู้ใช้อาจเจอเรื่องแปลก ๆ สามแบบ

ปัญหาแรก เขียนแล้วอ่านไม่เจอ

ผู้ใช้โพสต์ความคิดเห็น แล้วรีเฟรชหน้าทันที ถ้าการอ่านไปตกที่ผู้ตามที่ยังไม่ได้รับข้อมูล ความคิดเห็นก็หายไป ผู้ใช้อาจคิดว่าโพสต์ไม่สำเร็จ

สิ่งที่ต้องการคือ read-after-write consistency คือผู้ใช้ต้องเห็นสิ่งที่ตัวเองเพิ่งเขียนเสมอ วิธีทำ เช่น

- ข้อมูลที่ผู้ใช้แก้ได้เอง เช่น โปรไฟล์ของตัวเอง ให้อ่านจากผู้นำเสมอ
- ภายในหนึ่งนาทีหลังจากผู้ใช้แก้ข้อมูล ให้อ่านจากผู้นำ
- ให้ฝั่งผู้ใช้จำเวลาที่เขียนล่าสุด แล้วอ่านเฉพาะจากสำเนาที่อัปเดตถึงเวลานั้นแล้ว

ถ้าผู้ใช้ใช้หลายอุปกรณ์ เช่น เขียนจากคอมพิวเตอร์แล้วอ่านจากมือถือ เรื่องนี้จะยากขึ้นอีก

ปัญหาที่สอง ข้อมูลย้อนเวลา

ผู้ใช้รีเฟรชครั้งแรก เห็นความคิดเห็นใหม่ รีเฟรชอีกครั้ง ความคิดเห็นนั้นหายไป เพราะครั้งแรกอ่านจากสำเนาที่อัปเดตแล้ว ครั้งที่สองอ่านจากสำเนาที่ยังล้าหลัง

สิ่งที่ต้องการคือ monotonic reads คือเมื่อเห็นข้อมูลใหม่แล้ว ต้องไม่เห็นข้อมูลที่เก่ากว่านั้นอีก วิธีง่าย ๆ คือให้ผู้ใช้แต่ละคนอ่านจากสำเนาเดิมเสมอ

ปัญหาที่สาม เห็นคำตอบก่อนคำถาม

หนังสือยกตัวอย่างบทสนทนา คุณพูนส์ถามว่า คุณเค้ก คุณมองเห็นอนาคตได้ไกลแค่ไหน คุณเค้กตอบว่า ปกติก็ประมาณสิบวินาทีค่ะ คุณพูนส์

แต่ถ้าคำตอบของคุณเค้กไปถึงสำเนาที่ผู้สังเกตการณ์อ่านก่อนคำถาม ผู้สังเกตการณ์จะเห็นคำตอบก่อนคำถาม ราวกับคุณเค้กมองเห็นอนาคตจริง ๆ

สิ่งที่ต้องการคือ consistent prefix reads คือถ้าข้อมูลถูกเขียนตามลำดับไหน คนอ่านก็ต้องเห็นตามลำดับนั้น ปัญหานี้เกิดบ่อยในฐานข้อมูลที่แบ่งข้อมูลไว้หลายส่วน ซึ่งเราจะพูดถึงในบทหน้า

ผู้เขียนแนะนำว่า ตอนออกแบบระบบ ควรถามตัวเองว่า ถ้าสำเนาล้าหลังไปหลายนาที ระบบจะเป็นอย่างไร ถ้าผลลัพธ์แย่ต่อผู้ใช้ ก็ต้องออกแบบให้รับมือได้ ไม่ควรแกล้งทำเป็นว่าการทำสำเนาเป็นแบบรอ ทั้งที่จริงเป็นแบบไม่รอ

ตอนที่ 6: แบบมีผู้นำหลายคน

แบบผู้นำคนเดียวมีจุดอ่อนคือ ถ้าติดต่อผู้นำไม่ได้ ก็เขียนข้อมูลไม่ได้เลย จึงมีแนวคิดให้มีผู้นำหลายคน ผู้นำแต่ละคนรับการเขียนได้ แล้วส่งต่อให้ผู้นำคนอื่น เรียกว่า multi-leader

ใช้ในสถานการณ์ไหน

สถานการณ์แรก คือระบบที่มีศูนย์ข้อมูลหลายแห่ง แต่ละแห่งมีผู้นำของตัวเอง การเขียนไม่ต้องข้ามทวีปไปหาผู้นำคนเดียว จึงเร็วขึ้น ถ้าศูนย์ข้อมูลแห่งหนึ่งล่ม แห่งอื่นก็ทำงานต่อได้ และทนต่อปัญหาเครือข่ายระหว่างศูนย์ได้ดีกว่า

สถานการณ์ที่สอง คือแอปที่ใช้งานแบบออฟไลน์ได้ เช่น แอปปฏิทินที่มีทั้งในมือถือ แท็บเล็ต และคอมพิวเตอร์ แต่ละอุปกรณ์ก็เหมือนศูนย์ข้อมูลขนาดจิ๋วที่มีผู้นำของตัวเอง แก้ข้อมูลได้แม้ไม่มีอินเทอร์เน็ต แล้วค่อยซิงก์กันทีหลัง

สถานการณ์ที่สาม คือการแก้เอกสารร่วมกันแบบ Google Docs ซึ่งหลายคนแก้พร้อมกันได้

ปัญหาใหญ่ คือการเขียนชนกัน

สมมติหน้าวิกิหน้าหนึ่งมีชื่อว่า A ผู้ใช้คนแรกแก้ชื่อเป็น B ผ่านผู้นำคนหนึ่ง ผู้ใช้คนที่สองแก้ชื่อเป็น C ผ่านผู้นำอีกคนหนึ่ง ในเวลาเดียวกัน ทั้งสองการแก้ไขสำเร็จ แต่พอผู้นำส่งข้อมูลให้กัน ก็พบว่าขัดแย้งกัน

วิธีรับมือมีหลายแบบ

วิธีที่ดีที่สุดคือหลีกเลี่ยงตั้งแต่ต้น เช่น ให้ผู้ใช้แต่ละคนเขียนผ่านผู้นำคนเดิมเสมอ

ถ้าเลี่ยงไม่ได้ ต้องทำให้ทุกสำเนาได้ผลลัพธ์เดียวกันในที่สุด เช่น

- ให้การเขียนที่มีเวลาล่าสุดชนะ เรียกว่า last write wins หรือ LWW ง่ายแต่ทำให้ข้อมูลหายได้
- รวมค่าทั้งสองเข้าด้วยกัน เช่น ตั้งชื่อเป็น B และ C
- เก็บข้อขัดแย้งไว้ แล้วให้โค้ดของแอปหรือผู้ใช้ตัดสินใจทีหลัง

นอกจากนี้ยังมีงานวิจัยเรื่องการแก้ข้อขัดแย้งอัตโนมัติ เช่น โครงสร้างข้อมูลชื่อ CRDT ที่รวมการแก้ไขพร้อมกันได้เองอย่างสมเหตุสมผล และเทคนิค operational transformation ที่ Google Docs ใช้

ข้อขัดแย้งบางแบบก็มองเห็นยาก เช่น ระบบจองห้องประชุม ถ้าสองคนจองห้องเดียวกันเวลาเดียวกันผ่านผู้นำคนละคน ทั้งสองการจองดูถูกต้องเมื่อแยกกันดู แต่พอรวมกันกลับผิด

รูปแบบการเชื่อมต่อระหว่างผู้นำ

ผู้นำหลายคนเชื่อมกันได้หลายแบบ เช่น ต่อกันเป็นวงกลม ต่อแบบดาวโดยมีผู้นำตรงกลางคอยส่งต่อ หรือให้ทุกคนเชื่อมกับทุกคน

แบบวงกลมและแบบดาวมีจุดอ่อนคือ ถ้าเครื่องหนึ่งเสีย การส่งข้อมูลอาจขาดตอน ส่วนแบบทุกคนเชื่อมกับทุกคนทนทานกว่า แต่ข้อความอาจไปถึงไม่ตรงลำดับ เช่น คำสั่งแก้ไขแถวหนึ่งไปถึงก่อนคำสั่งสร้างแถวนั้น

ผู้เขียนสรุปว่า แบบผู้นำหลายคนมักเป็นของที่ติดตั้งเพิ่มเติมภายหลัง และมีกับดักซ่อนอยู่เยอะ ควรหลีกเลี่ยงถ้าทำได้

ตอนที่ 7: แบบไม่มีผู้นำ

แนวคิดสุดท้ายคือ ไม่มีผู้นำเลย ทุกสำเนารับการเขียนได้หมด แนวคิดนี้กลับมาได้รับความนิยมเพราะระบบ Dynamo ของ Amazon จึงมักเรียกว่าแบบ Dynamo ฐานข้อมูลที่ใช้ เช่น Cassandra, Riak และ Voldemort

เขียนข้อมูลขณะที่บางเครื่องเสีย

สมมติมีสำเนาสามเครื่อง และเครื่องหนึ่งดับอยู่ ผู้ใช้ส่งการเขียนไปทั้งสามเครื่องพร้อมกัน สองเครื่องตอบรับ ก็ถือว่าเขียนสำเร็จ

พอเครื่องที่ดับกลับมา มันก็มีข้อมูลเก่า ดังนั้นเวลาอ่าน ผู้ใช้ต้องอ่านจากหลายเครื่องพร้อมกัน แล้วดูหมายเลขเวอร์ชันว่าค่าไหนใหม่ที่สุด

ทำให้เครื่องที่ล้าหลังตามทัน

มีสองวิธี วิธีแรกคือ read repair ตอนอ่าน ถ้าพบว่าเครื่องไหนมีค่าเก่า ก็เขียนค่าใหม่กลับไปให้เลย วิธีที่สองคือมีงานเบื้องหลังคอยเทียบข้อมูลระหว่างเครื่อง แล้วคัดลอกส่วนที่ขาดไปให้ เรียกว่า anti-entropy

กฎเสียงข้างมาก หรือ Quorum

ถ้ามีสำเนา n เครื่อง การเขียนต้องสำเร็จอย่างน้อย w เครื่อง และการอ่านต้องได้คำตอบอย่างน้อย r เครื่อง ถ้า w บวก r มากกว่า n ก็มั่นใจได้ว่าในเครื่องที่เราอ่าน จะมีอย่างน้อยหนึ่งเครื่องที่มีค่าล่าสุด เพราะสองกลุ่มต้องซ้อนทับกันอย่างน้อยหนึ่งเครื่อง

ตัวอย่างที่ใช้บ่อย คือ n เท่ากับสาม w เท่ากับสอง และ r เท่ากับสอง ระบบนี้ทนได้ถ้าเครื่องเสียหนึ่งเครื่อง ถ้ามีห้าเครื่อง โดย w และ r เท่ากับสาม ก็ทนได้ถึงสองเครื่อง

ถ้างานส่วนใหญ่เป็นการอ่าน ก็ตั้ง r ต่ำ w สูงได้ การอ่านจะเร็ว แต่การเขียนจะล้มเหลวง่ายขึ้น

ข้อจำกัดของ Quorum

ฟังดูเหมือนรับประกันว่าจะได้ค่าล่าสุดเสมอ แต่ในความจริงมีกรณีขอบหลายแบบที่ยังได้ค่าเก่า เช่น

- มีการเขียนสองครั้งพร้อมกัน แล้วไม่รู้ว่าครั้งไหนมาก่อน
- การอ่านเกิดขึ้นระหว่างที่การเขียนยังไปไม่ครบทุกเครื่อง
- การเขียนสำเร็จบางเครื่องแต่ไม่ถึงจำนวนที่กำหนด ระบบรายงานว่าล้มเหลว แต่เครื่องที่เขียนไปแล้วก็ไม่ได้ย้อนกลับ
- เครื่องที่มีค่าใหม่เสีย แล้วถูกกู้คืนจากเครื่องที่มีค่าเก่า

ดังนั้น ควรมองว่า quorum ช่วยเพิ่มโอกาสได้ค่าล่าสุด ไม่ใช่การรับประกันแบบเด็ดขาด

นอกจากนี้ การตรวจว่าข้อมูลล้าหลังไปแค่ไหน ทำได้ยากกว่าแบบมีผู้นำ เพราะไม่มีลำดับการเขียนที่แน่นอนให้เทียบ

Sloppy Quorum และ Hinted Handoff

ถ้าเครือข่ายมีปัญหาจนติดต่อเครื่องที่ควรเก็บข้อมูลไม่ได้ครบ ระบบบางตัวยอมให้เขียนลงเครื่องอื่นไปก่อนชั่วคราว พอเครื่องเดิมกลับมา ค่อยส่งข้อมูลคืนให้

ผู้เขียนเปรียบเทียบว่า เหมือนลืมกุญแจบ้าน เลยไปขออาศัยนอนโซฟาบ้านเพื่อนบ้านก่อน พอได้กุญแจแล้วค่อยกลับบ้าน

วิธีนี้ช่วยให้เขียนข้อมูลได้แม้เครือข่ายมีปัญหา แต่ก็แลกกับการที่ quorum ไม่ได้รับประกันอะไรเลยในช่วงนั้น

ตอนที่ 8: การจัดการการเขียนพร้อมกัน

ในระบบที่ไม่มีผู้นำ หรือมีผู้นำหลายคน ข้อมูลอาจถูกเขียนพร้อมกันจากหลายที่ และไปถึงแต่ละเครื่องในลำดับต่างกัน ถ้าแต่ละเครื่องแค่เขียนทับด้วยค่าที่มาถึงล่าสุด ข้อมูลในแต่ละเครื่องก็จะไม่ตรงกันไปตลอด

ให้ค่าล่าสุดชนะ

วิธีง่ายที่สุดคือแนบเวลาไปกับการเขียน แล้วเก็บค่าที่มีเวลาล่าสุดไว้ Cassandra ใช้วิธีนี้

ปัญหาคือ ข้อมูลอื่นถูกทิ้งไปเงียบ ๆ แม้ผู้ใช้จะได้รับแจ้งว่าเขียนสำเร็จแล้วก็ตาม วิธีที่ปลอดภัยคือให้แต่ละคีย์ถูกเขียนแค่ครั้งเดียว แล้วไม่แก้อีกเลย เช่น ใช้รหัสสุ่มที่ไม่ซ้ำกันเป็นคีย์

อะไรคือการเขียนพร้อมกัน

ผู้เขียนอธิบายว่า การเขียนพร้อมกันไม่ได้แปลว่าเกิดในเวลาเดียวกันตามนาฬิกา แต่หมายถึงการเขียนสองครั้งที่ต่างฝ่ายต่างไม่รู้ว่าอีกฝ่ายเกิดขึ้น

ถ้าการเขียนครั้งที่สองรู้จักค่าจากครั้งแรก และต่อยอดจากค่านั้น ก็ถือว่าครั้งแรกเกิดก่อน ความสัมพันธ์นี้เรียกว่า happens-before แบบนี้ครั้งที่สองแทนที่ครั้งแรกได้อย่างปลอดภัย

แต่ถ้าต่างคนต่างไม่รู้จักกัน ก็ถือว่าพร้อมกัน และต้องหาทางรวมค่าเข้าด้วยกัน

ตัวอย่างตะกร้าสินค้า

หนังสือยกตัวอย่างลูกค้าสองคนที่ใช้ตะกร้าสินค้าเดียวกัน คนแรกใส่นม อีกคนใส่ไข่ในเวลาไล่เลี่ยกัน

ระบบแก้ด้วยหมายเลขเวอร์ชัน

- ทุกครั้งที่เขียน เซิร์ฟเวอร์จะเพิ่มหมายเลขเวอร์ชัน แล้วส่งหมายเลขนั้นกลับไปให้ผู้ใช้
- ก่อนเขียน ผู้ใช้ต้องอ่านค่าก่อน แล้วส่งหมายเลขเวอร์ชันที่ตัวเองเห็นกลับมาพร้อมการเขียน
- เซิร์ฟเวอร์จะรู้ว่าผู้ใช้เห็นข้อมูลถึงเวอร์ชันไหน ค่าที่เก่ากว่านั้นแทนที่ได้ แต่ค่าที่ใหม่กว่า ซึ่งผู้ใช้ยังไม่เคยเห็น ต้องเก็บไว้คู่กัน

ค่าที่เก็บไว้คู่กันเรียกว่า sibling หรือพี่น้อง โค้ดของแอปต้องรวมค่าเหล่านี้เอง สำหรับตะกร้าสินค้า วิธีที่สมเหตุสมผลคือเอาสินค้าจากทุกค่ามารวมกัน ตะกร้าก็จะมีทั้งนมและไข่

แต่ถ้ามีการลบสินค้าออก การรวมแบบนี้จะทำให้สินค้าที่ลบไปแล้วกลับมาใหม่ จึงต้องทิ้งเครื่องหมายไว้ว่าสินค้านี้ถูกลบแล้ว ซึ่งก็คือ tombstone ที่เราเจอในบทที่สาม

Version Vector

ถ้ามีหลายสำเนา หมายเลขเวอร์ชันตัวเดียวไม่พอ ต้องให้แต่ละสำเนามีหมายเลขของตัวเอง แล้วเก็บหมายเลขของทุกสำเนาไว้เป็นชุด เรียกว่า version vector ชุดหมายเลขนี้ช่วยให้รู้ว่าการเขียนไหนเกิดก่อน และการเขียนไหนเกิดพร้อมกัน

ตอนที่ 9: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง เราทำสำเนาข้อมูลเพื่อให้ระบบทำงานต่อได้แม้มีเครื่องเสีย ใช้งานได้แม้เครือข่ายขาด ข้อมูลอยู่ใกล้ผู้ใช้ และรับการอ่านได้มากขึ้น

สอง แบบมีผู้นำคนเดียว เข้าใจง่ายและไม่มีการเขียนชนกัน จึงใช้กันมากที่สุด แต่การเปลี่ยนผู้นำเมื่อผู้นำเสียมีกับดักหลายข้อ

สาม แบบมีผู้นำหลายคน และแบบไม่มีผู้นำ ทนทานกว่าเมื่อเครื่องเสียหรือเครือข่ายช้า แต่ต้องรับมือกับการเขียนชนกันและรับประกันความถูกต้องได้น้อยกว่า

สี่ การทำสำเนาแบบไม่รอนั้นเร็ว แต่ทำให้สำเนาล้าหลังได้ ผู้ใช้อาจเจอปัญหาเขียนแล้วอ่านไม่เจอ ข้อมูลย้อนเวลา หรือเห็นคำตอบก่อนคำถาม ต้องออกแบบรับมือให้เหมาะกับแอปของเรา

ห้า การเขียนพร้อมกันไม่ได้ดูจากนาฬิกา แต่ดูจากว่าการเขียนแต่ละครั้งรู้จักกันหรือไม่ และหมายเลขเวอร์ชันคือเครื่องมือสำคัญที่ใช้แยกแยะเรื่องนี้

บทต่อไปคือบทที่หก ว่าด้วยการแบ่งข้อมูลเป็นส่วน ๆ หรือ Partitioning สำหรับกรณีที่ข้อมูลใหญ่เกินจะเก็บไว้ในเครื่องเดียว ขอบคุณที่ฟังครับ

ทิปสำหรับ Edge: บทนี้มีตัวอักษร n, w, r ซึ่งเสียงไทยของ Edge อาจอ่านเป็นภาษาอังกฤษปนกันไปบ้าง ถ้าฟังแล้วงง จำแค่หลักว่า "จำนวนเครื่องที่เขียนสำเร็จ บวกกับจำนวนเครื่องที่อ่าน ต้องมากกว่าจำนวนสำเนาทั้งหมด" ก็พอครับ ส่วนถ้าจะแบ่งฟังสองรอบ แนะนำรอบแรกตอนที่ 1–5 (แบบมีผู้นำคนเดียว) และรอบสองตอนที่ 6–9 ครับ

จะให้ทำบทที่ 6 ต่อเลยไหมครับ หรือจะรวมบทที่ 1–5 เป็นไฟล์ PDF ไว้เปิดใน Edge ก็ได้ 😊